Property Price Prediction

Intelligent Real Estate Valuation & Investment Grade Classification

via Feature Engineering and XGBoost

Project 9 — Milestone 1 Report

GenAI Project — Machine Learning Pipeline & Streamlit Application

GitHub Repository:
github.com/Jag2007/property-price-prediction

Live Application:
property-price-prediction-real-estate.streamlit.app

Date: March 1, 2026

Version: 1.0

Abstract

This report presents an end-to-end machine learning system for intelligent real estate valuation and investment grade classification. The pipeline operates on a dataset of 1,800 property records, applying robust preprocessing (median imputation, ordinal and one-hot encoding, IQR-based outlier clipping), and trains two XGBoost models: a regressor for predicting `Current_Market_Price` and a multi-class classifier for predicting `Investment_Grade` (classes 0, 1, 2). The regression model achieves an R^2 of **0.9483** with an RMSE of ≈1.01M, while the classifier reaches **97.50**% accuracy and a weighted F1-score of **0.97**. A production-ready Streamlit application supports both manual and batch (CSV) inference and is publicly deployed on Streamlit Community Cloud.

Contents

1 Introduction

1.1 Motivation

Real estate represents one of the most significant investment categories globally. Two persistent challenges face buyers, sellers, and investors:

1. **Manual valuation inconsistencies** — property pricing varies non-linearly with feature interactions (location, size, amenities, market dynamics), making human appraisals costly and error-prone.
2. **Investment grade ambiguity** — classifying a property as a good investment requires simultaneously considering price, yield, and contextual signals that are difficult to synthesise manually.

1.2 Objective

The *Property Price Prediction* system is a production-structured machine learning pipeline that:

1. Accepts raw property attributes as input.
2. Applies a comprehensive preprocessing and feature engineering pipeline.
3. Predicts a continuous `Current_Market_Price` via XGBoost regression.
4. Classifies a property into `Investment_Grade` $\in \{0, 1, 2\}$ via XGBoost classification.
5. Exposes predictions through an interactive, publicly deployed Streamlit application.

1.3 Scope

- **Dataset:** 1,800 residential property records with 17 raw columns.
- **Tasks:** Regression (price) and multi-class classification (investment grade).
- **Models:** XGBoost Regressor and XGBoost Classifier.
- **Deployment:** Streamlit Community Cloud (public, live).

2 Problem Statement

The core challenges addressed by this project are summarised below:

Table 1: Problem Scope and Challenges

Challenge	Scope
Manual valuation inconsistencies	Property pricing varies by feature interactions
Multi-objective prediction	Continuous price + categorical investment grade
Data quality issues	Missing values, categorical features, outliers
Deployment requirement	Real-time predictions for unseen properties

3 Dataset

3.1 Source and Scale

The dataset `real_estate_raw.csv` comprises **1,800 property records** with **17 columns**, as confirmed by the training notebook:

Listing 1: Dataset Loading

```
df = pd.read_csv("real_estate_raw.csv")
print("Shape of dataset:", df.shape)
# Output: Shape of dataset: (1800, 17)
```

Each record contains property attributes spanning structural, locational, environmental, and financial dimensions, together with two target variables:

- `Current_Market_Price` — continuous regression target.
- `Investment_Grade` — categorical classification target (0, 1, or 2).

3.2 Feature Overview

Table 2: Dataset Feature Summary

Feature	Type	Description
Total_Square_Footage	Numeric (Float)	Total property area
Bedrooms	Numeric (Int)	Number of bedrooms
Bathrooms	Numeric (Int)	Number of bathrooms
Age_of_Property	Numeric (Int)	Age in years
Floor_Number	Numeric (Int)	Floor level
Furnishing_Status	Categorical	Unfurnished / Semi / Fully
Neighborhood	Categorical	Locality identifier
Distance_to_City_Center_km	Numeric (Float)	Distance in km
Proximity_to_Public_Transport_km	Numeric (Float)	Transport proximity
Crime_Index	Numeric (Float)	Local crime rate
Air_Quality_Index	Numeric (Float)	Environmental quality
Neighborhood_Growth_Rate_%	Numeric (Float)	Growth rate
Price_per_SqFt	Numeric (Float)	Unit price
Annual_Property_Tax	Numeric (Float)	Tax value
Estimated_Rental_Yield_%	Numeric (Float)	Rental return
Current_Market_Price	Numeric (Float)	**Regression target**
Investment_Grade	Integer (0/1/2)	**Classification target**

4 Data Preprocessing Pipeline

A structured preprocessing pipeline was implemented to prepare the raw dataset for modelling. All steps are applied before any train/test split to prevent data leakage.

4.1 Missing Value Imputation

Numeric columns were imputed using **median values**, which is robust to the outliers present in real-estate data:

Listing 2: Median Imputation

```
num_cols = df.select_dtypes(include=np.number).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())
```

4.2 Categorical Encoding

Two encoding strategies were applied depending on feature ordinality:

Listing 3: Ordinal Encoding

Ordinal Encoding — `Furnishing_Status`

```
furnish_map = {
    'Unfurnished': 0,
    'Semi-furnished': 1,
    'Fully-furnished': 2
}
df['Furnishing_Status'] = df['Furnishing_Status'].map(furnish_map)
```

One-Hot Encoding — `Neighborhood` One-hot encoding with `drop_first=True` was applied to eliminate multicollinearity from the dummy variable trap:

Listing 4: One-Hot Encoding

```
df = pd.get_dummies(df, columns=['Neighborhood'], drop_first=True)
```

4.3 Outlier Treatment

Inter-Quartile Range (IQR) clipping was applied to all numeric columns *excluding* the two target variables (`Current_Market_Price` and `Investment_Grade`) to preserve their natural distributions:

Listing 5: IQR-Based Outlier Clipping

```
for col in num_cols:
    if col not in ['Current_Market_Price', 'Investment_Grade']:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df[col] = np.clip(df[col], lower, upper)
```

The cleaned dataset was persisted as `data/processed/real_estate_clean.csv` for reproducibility.

5 System Architecture

5.1 High-Level Pipeline

The system is composed of two independent pipelines that share the same preprocessed dataset:

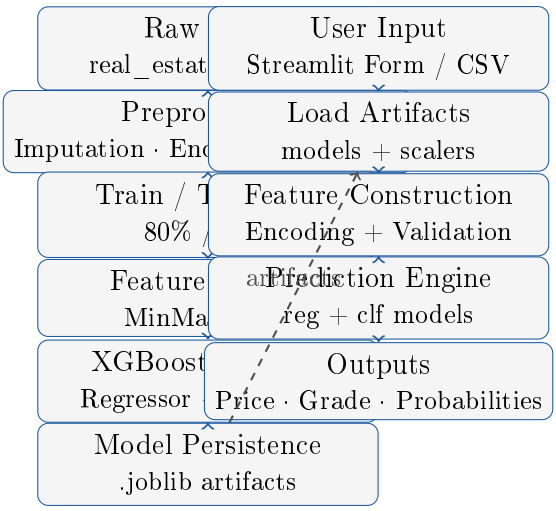

Figure 1: End-to-End System Architecture

5.2 Repository Structure

Listing 6: Project Directory Layout

```
property-price-prediction/
          README.md
          requirements.txt
          app/
                streamlit_app.py
          data/
                raw/
                      real_estate_raw.csv
                processed/
                    real_estate_clean.csv
          models/
              xgb_regression_model.joblib
              regression_scaler.joblib
              xgb_classification_model.joblib
              classification_scaler.joblib
```

6 Model Architecture

6.1 XGBoost Regression — `Current_Market_Price`

- **Target:** `Current_Market_Price` (continuous)
- **Features:** all engineered predictors, excluding `Investment_Grade`
- **Scaling:** `MinMaxScaler` (fit on training set only)

- **Split:** 80% train / 20% test (`random_state=42`)

Table 3: XGBRegressor Hyperparameters

Parameter	Value
`n_estimators`	200
`learning_rate`	0.05
`max_depth`	5
`random_state`	42
`objective`	`reg:squarederror` (default)

6.2 XGBoost Classification — `Investment_Grade`

- **Target:** `Investment_Grade` $\in \{0, 1, 2\}$
- **Features:** all engineered predictors, excluding both `Investment_Grade` and `Current_Market_Pri` (to prevent target leakage)
- **Scaling:** `MinMaxScaler` (separate scaler from regression)
- **Split:** 80% train / 20% test (`random_state=42`)

Table 4: XGBClassifier Hyperparameters

Parameter	Value
`n_estimators`	200
`learning_rate`	0.05
`max_depth`	4
`random_state`	42
`eval_metric`	`mlogloss`

7 Evaluation

7.1 Evaluation Metrics

7.1.1 Regression

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{1}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{2}$$

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} \tag{3}$$

7.1.2 Classification

Accuracy, weighted Precision, weighted Recall, and weighted F1-Score were computed via `sklearn.metrics.classification_report`.

7.2 Results

Table 5: Model Performance Summary

Metric	Regression (Price)	Classification (Grade)
R^2 Score	**0.9483**	—
MAE	**550,850.82**	—
RMSE	**1,010,636.01**	—
Accuracy	—	**97.50%**
Weighted Precision	—	**0.98**
Weighted Recall	—	**0.97**
Weighted F1-Score	—	**0.97**

7.3 Classification Report — Per-Class Analysis

Table 6: Per-Class Performance (Investment Grade Classification)

Class	Precision	Recall	F1-Score	Support
0 (Grade 0)	0.99	0.96	0.98	167
1 (Grade 1)	0.96	0.99	0.98	171
2 (Grade 2)	1.00	0.91	0.95	22
Accuracy		**0.975 (360 samples)**		
Macro avg	0.98	0.96	0.97	360
Weighted avg	0.98	0.97	0.97	360

7.4 Key Observations

1. **Regression:** An R^2 of 0.9483 demonstrates that XGBoost explains more than 94% of the variance in property prices. The relatively high RMSE (≈1.01M) reflects the wide price range inherent in the real-estate domain.
2. **Classification:** 97.5% accuracy with consistently high precision and recall across all three investment grade classes validates the classifier's robustness.
3. **Class 2 Recall:** The slightly lower recall for class 2 (0.91) compared to classes 0 and 1 is consistent with the lower support (22 samples), indicating a class imbalance concern for grade 2 properties.
4. **Leakage Prevention:** `Current_Market_Price` was explicitly excluded from the classification feature set to prevent target leakage, ensuring the model generalises to cold-start predictions.

8 Model Persistence and Inference

8.1 Artifact Saving

All models and scalers are serialised using `joblib` for consistent, efficient loading in the inference layer:

Listing 7: Artifact Serialisation

```
import joblib

joblib.dump(reg_model,     "xgb_regression_model.joblib")
joblib.dump(scaler_reg,    "regression_scaler.joblib")
joblib.dump(clf_model,     "xgb_classification_model.joblib")
joblib.dump(scaler_clf,    "classification_scaler.joblib")
```

8.2 Inference Pipeline

The Streamlit inference layer (`app/streamlit_app.py`) implements the following function sequence:

1. `load_artifacts(...)` — loads models and scalers from `models/`
2. `build_feature_row(...)` — constructs feature vector for manual input
3. `raw_to_feature_frame(...)` — transforms raw CSV rows for batch inference
4. `run_predict(...)` — scales features and calls both predict heads
5. `page_manual(...)` / `page_csv(...)` — UI page handlers

8.3 Sample Inference

A representative sample prediction from the training notebook:

Listing 8: Sample Prediction

```
sample = X_test.iloc[[78]]
predicted_price = reg_model.predict(scaler_reg.transform(sample))
    [0]

sample_clf = X_test_c.iloc[[0]]
predicted_grade = clf_model.predict(scaler_clf.transform(sample_clf
    ))[0]

# Output:
# Predicted Price: 16,749,228.0
# Predicted Investment Grade: 1
```

9 Application Interface

The Streamlit application provides three pages:

9.1 Predict (Manual Input)

Users fill in a structured form with individual property attributes. On submission, both `Current_Market_Price` and `Investment_Grade` are predicted simultaneously, along with a class probability distribution bar chart.

9.2 Batch Predict (CSV Upload)

Accepts a CSV file for bulk inference. Supports:

- Fully pre-encoded feature columns, or
- Raw columns with `Furnishing_Status` and `Neighborhood` in original string form.

Predictions are available as a downloadable CSV.

9.3 About

A documentation page describing the preprocessing methodology, model architecture, and deployment workflow.

9.4 Deployment

The application is publicly hosted on **Streamlit Community Cloud**:

`property-price-prediction-real-estate.streamlit.app`

10 Limitations

1. **Dataset Size:** With 1,800 records, the model behaviour is tightly coupled to the training distribution. Out-of-distribution properties (extreme sizes, unusual locations) may yield less reliable predictions.
2. **Investment Grade Semantics:** The grade labels (0, 1, 2) are numeric class IDs. Business meaning (e.g., Poor / Good / Excellent) should be mapped explicitly in a production advisory system.
3. **Market Drift:** Real-world price dynamics change continuously. The static model will degrade over time without periodic retraining on fresh data.
4. **Class Imbalance (Grade 2):** The low support for class 2 (22 test samples) reduces confidence in recall estimates for that grade category.
5. **Feature Interactions:** Complex non-linear location-price interactions (micro-market dynamics, zoning changes) are not captured by the current feature set.

11 Future Work

11.1 Short-Term Improvements (Milestone 1+)

- Apply SMOTE or class-weight balancing for Investment Grade class 2.
- Use SHAP values for feature importance interpretability.
- Expand feature engineering with location-intelligence signals (transport score, nearby amenity index, pincode zone index).
- Implement model monitoring for input and prediction drift.

11.2 Milestone 2 — Agentic AI Framework

- **LangGraph Agent:** Autonomous reasoning over property characteristics to generate structured investment advisory reports.

- **RAG Integration:** Retrieval of real-time market trends, regulations, and comparable listings via Chroma/FAISS.
- **Structured Advisory Outputs:** Buy/Hold/Avoid recommendations with risk notes.
- **Explainability Layer:** Feature contribution view per prediction to justify price and grade estimates.

12 Conclusion

This project demonstrates that gradient-boosted tree models, combined with principled feature engineering and preprocessing, can deliver high-quality automated real estate valuation and investment grade classification. The XGBoost regression model achieves $R^2 = 0.9483$ on held-out test data, while the XGBoost classifier reaches 97.5% accuracy with strong per-class precision and recall.

The modular architecture—separate preprocessing, model artifacts, and Streamlit application—enables straightforward extension toward the Milestone 2 agentic framework. Key next steps include addressing class imbalance for grade 2 properties, incorporating richer location features, and building the LangGraph-based advisory agent.

A Technology Stack

Table 7: Technology Stack Summary

Component	**Technology**
Language	Python 3.10+
ML Framework	XGBoost $\geq$ 2.0, scikit-learn
Data Processing	Pandas $\geq$ 2.0, NumPy
Web Framework	Streamlit
Serialisation	Joblib
Hosting	Streamlit Community Cloud
Version Control	Git / GitHub

B Input–Output Specification

Table 8: Input Feature Vector

#	Feature Name	Type
1	Total_Square_Footage	Float
2	Bedrooms	Integer
3	Bathrooms	Integer
4	Age_of_Property	Integer
5	Floor_Number	Integer
6	Furnishing_Status	Ordinal (0/1/2)
7	Distance_to_City_Center_km	Float
8	Proximity_to_Public_Transport_km	Float
9	Crime_Index	Float
10	Air_Quality_Index	Float
11	Neighborhood_Growth_Rate_%	Float
12	Price_per_SqFt	Float
13	Annual_Property_Tax	Float
14	Estimated_Rental_Yield_%	Float
15	Neighborhood_* (one-hot)	Binary

Regression Output: `Current_Market_Price` (Float)
Classification Output: `Investment_Grade` $\in \{0, 1, 2\}$ + class probabilities